PLAN FOR TODAY

1. By the end of today I can...

- [ ] log in with `ssh lab` (a nickname)
- [ ] log in with an SSH key, no password
- [ ] keep a long job running with tmux
- [ ] copy files both ways: `scp` and an SFTP program
- [ ] sync code and notes with git
- [ ] open the Pi desktop with VNC
- [ ] add a second Wi-Fi to a Pi; find a Pi when `.local` fails
- [ ] choose a safe way to reach a Pi in another network

My 2 hours

A review + plan	10
B smarter SSH	25
C moving files	25
D remote desktop	25
E more networks	20
F security	15

my notes

2. Day 1 rough edges, today's remedies

Rough edge	Remedy	Part
long command to type	nickname in the SSH configuration file	B
password every time	an SSH key pair	B
job dies when connection drops	tmux	B
no way to move files	`scp`, an SFTP program, git	C
only a command line	VNC: the desktop in a window	D
one Wi-Fi network only	more than one Wi-Fi profile on the Pi	E
laptop and Pi on different networks	remote-access options (and what to avoid)	E, F

my notes

3. A naming plan for several Pis

Item	Suggestion	Example
host name	`raspi-` + place or owner	`raspi-lab`, `raspi-home`
user name	the same on every Pi	`piuser`
SSH nickname	short, easy to type	`lab`, `home`
connection card	one per Pi	board, name, IP, Wi-Fi

Rename a Pi: `sudo raspi-config` -> System Options -> Hostname -> reboot.

my notes

B. A NICKNAME FOR EACH PI

4. The SSH configuration file

- A small text file where I describe each machine once. Then `ssh`, `scp` and other tools accept the nickname.

```
# I create the folder and an empty file named exactly "config" (no .txt), open it
New-Item -ItemType Directory -Force $env:USERPROFILE\.ssh | Out-Null
New-Item -ItemType File -Force $env:USERPROFILE\.ssh\config | Out-Null
notepad $env:USERPROFILE\.ssh\config
```

2 type one block per Pi (indent with spaces) - file: C:\Users\me\.ssh\config

```
Host lab
    HostName raspi-lab.local
    User piuser

Host home
    HostName raspi-home.local
    User piuser

Host *
    ServerAliveInterval 60
```

```
# 3 test: only the nickname
ssh lab
```

`Host lab`	starts a block; `lab` is the nickname
`HostName`	the real destination: a `.local` name or an IP address (IP if `.local` is unreliable)
`User`	who I log in as; no need for `piuser@`
`Host *` / `ServerAliveInterval 60`	for all hosts: a signal every 60 s keeps idle connections open

Mac / Linux: `~/.ssh/config`, same contents (`mkdir -p ~/.ssh; nano ~/.ssh/config`).

my notes

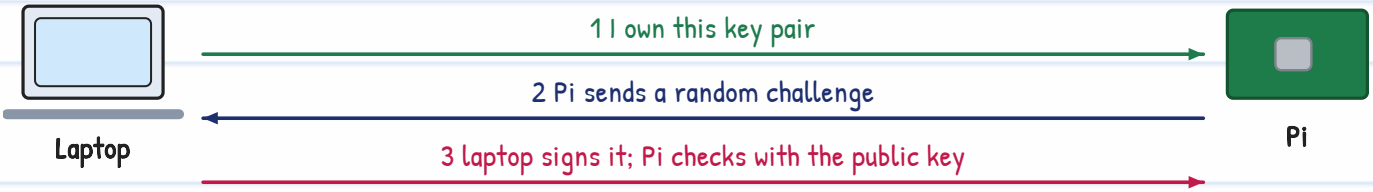
B. SSH KEYS

5. Log in without a password

- A password can be guessed, mistyped or stolen. A **key pair** is stronger: a **private key** (secret, stays on the laptop) and a **public key** (harmless to share) placed on the Pi.

private key
secret: never leaves the laptop

public key
~/.ssh/authorized_keys (shareable)



access without sending a secret over the network

```
# 1 create the pair on the laptop (Enter = default file; passphrase optional but protective)
ssh-keygen -t ed25519 -C "office-laptop"
# creates id_ed25519 (private, secret) and id_ed25519.pub (public) in .ssh
```

```
# 2 send the PUBLIC key to the Pi (asks the password one last time)
type $env:USERPROFILE\.ssh\id_ed25519.pub | ssh lab "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
# Mac / Linux: ssh-copy-id lab
# 3 test
ssh lab
```

Rules: never copy the private key anywhere. **One key per laptop**: with two laptops, create a key on each and add **both** public keys to each Pi. A re-written card loses its `authorized_keys` : repeat step 2. Keep password login on until the key works.

my notes

B. TMUX AND SMALL HABITS

6. tmux: sessions that survive a dropped connection

- A long job over SSH (download, training) normally dies with the terminal if the connection drops. **tmux** keeps the terminal alive on the Pi.

Without tmux

```
training... epoch 12/50
<connection drops>
-> the job is killed
```

With tmux

```
tmux new -s train
training... epoch 12/50
<connection drops>
-> keeps running
tmux attach -t train
```

```
sudo apt install tmux  # once
tmux new -s work  # start a session named work
# run the long command inside it; press Ctrl+b then d to DETACH
tmux ls  # list sessions
tmux attach -t work  # come back, from any laptop
exit  # inside tmux: end the session
```

Press Ctrl+b, then	Does
`d`	detach: leave the session running
`[`	scroll with the arrows; `q` to leave
`c` / `n` / `p`	new window / next / previous

my notes

7. Small habits that save time

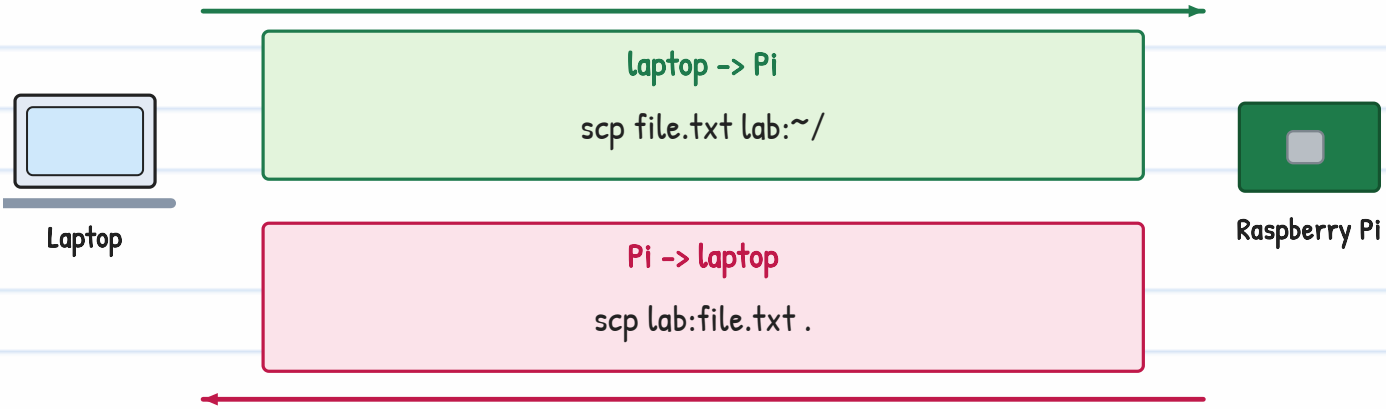
Up arrow	recall previous commands
Tab while typing a name	complete file and folder names
`history`	list past commands
Ctrl+C	stop the running command
`clear` or Ctrl+L	clean the screen
`exit` or Ctrl+D	leave the SSH session

my notes

C. MOVING FILES WITH SCP

8. scp: copy over SSH

- Same encrypted channel as SSH: nothing extra on the Pi. Pattern: **scp SOURCE DESTINATION.** `name:path` = on the Pi; a plain path = on this computer.



scp SOURCE DESTINATION 'name:path' = on the Pi; plain path = on this computer

Practice (10 min)

```
# 1 a test file on the laptop (in a writable folder)
"Hello from my laptop" | Out-File test.txt -Encoding ascii
# 2 send it (the colon is required; ~/ = my home folder on the Pi)
scp test.txt lab:~/
# 3 check on the Pi
ssh lab
ls -l ; cat test.txt ; cp test.txt test_2.txt ; exit
# 4 fetch a file (the dot = the current folder here)
scp lab:test_2.txt .
```

Windows paths: open PowerShell in the folder that holds the file and use short names. A path like `C:\Users\me\f.txt` contains a colon that can confuse scp about what is local.

my notes

C. MORE SCP

9. More scp

Task	Command	Notes
a folder	`scp -r photos lab:~/images/`	`-r` = recursive; destination must exist
create a folder first	`ssh lab "mkdir -p ~/images"`	scp never creates folders
several files	`scp a.jpg b.jpg lab:~/images/`	or `scp *.jpg lab:~/images/`
wildcard from the Pi	`scp "lab:~/images/*.jpg" .`	quote it so the Pi expands the `*`
spaces in names	`scp "my file.txt" lab:~/`	quote, or avoid spaces
another port	`scp -P 2222 test.txt lab:~/`	capital P for scp (ssh: lowercase p)

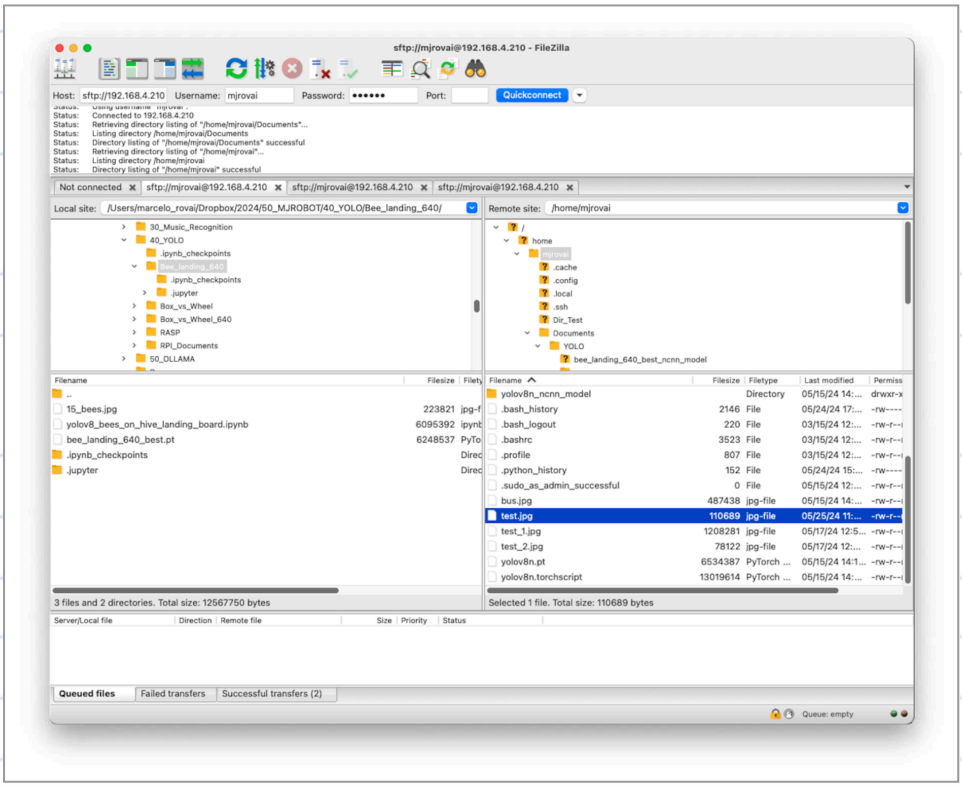
my notes

10. SFTP program with two panes

- Drag and drop: **FileZilla** or **WinSCP** (free; download from the official site and decline extra software).

Field	Value
Host	`sftp://raspi-lab.local` (or the IP)
Protocol (WinSCP)	SFTP
User / password	my Pi user and password (or a key file)
Port	22 (blank = default)

Press Quickconnect (FileZilla) or Login (WinSCP); accept the unknown-host message the first time; save the connection in the site manager. Left pane = laptop, right pane = Pi.



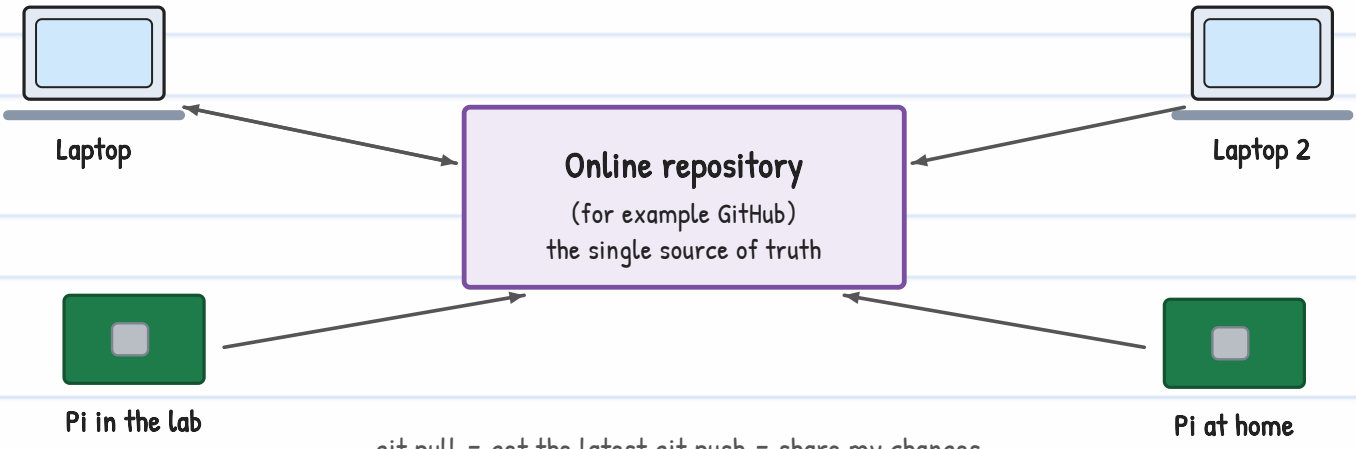
FileZilla connected to a Pi (example on a Mac)

my notes

C. SYNC WITH GIT; WHICH TOOL?

11. Several computers in sync with git

- With two laptops and two Pis, hand copying gets confusing. Keep **code and notes in an online repository** and let every computer sync with it.



git pull = get the latest git push = share my changes

```
sudo apt install git   # once, on the Pi
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git clone https://github.com/YOU/REPO.git   # first time
git pull   # get the latest
git add . ; git commit -m "Describe the change" ; git push   # share
```

Signing in to GitHub: GitHub no longer accepts the account password for `git push` over HTTPS. Create a **personal access token** (or an SSH key for the Pi) and give the token when git asks for a password. Keep it private.

Not in git: large files (models of hundreds of MB, datasets, videos). Use a `.gitignore` and move them with scp or SFTP.

my notes

12. Which tool when?

Tool	Best for	Remember
`scp`	a few files or a folder, scripts	quick to type
SFTP program	browsing, many files, drag and drop	save the connection
git	code and notes shared by computers	history; not for big files
USB drive	very large data on a slow network	remove only after `sync` or shutdown

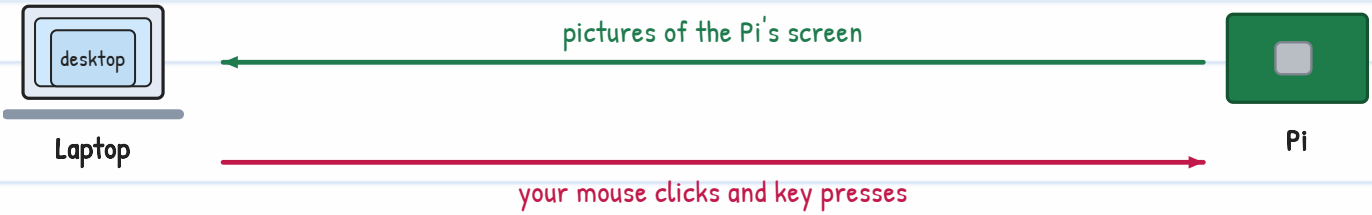
my notes

D. THE PI DESKTOP IN A WINDOW

13. SSH or desktop?

	SSH	VNC
sees	text only	the whole desktop, windows, mouse
load on the Pi	very small	noticeably more
network	very little	much more bandwidth
best for	install, run, edit, automate	browser, images, plots, graphical tools
needs full OS image	no (Lite works)	yes: the Pi runs a desktop

VNC = Virtual Network Computing: remote control of a graphical desktop

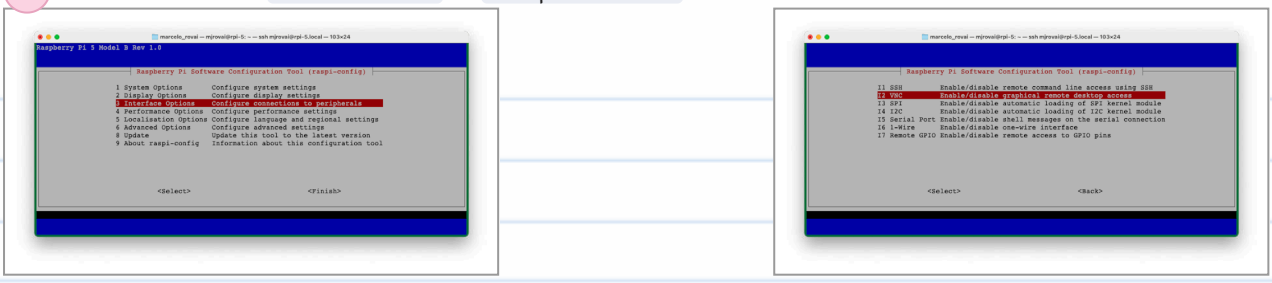


SSH carries text only; VNC carries the whole desktop picture

my notes

14. Turn on VNC on the Pi

1. `sudo raspi-config`
2. Interface Options -> **VNC** -> Yes; finish; reboot if offered (desktop: Raspberry Pi Configuration, Interfaces tab).
3. Know the address: `hostname -I` or `raspi-lab.local`.



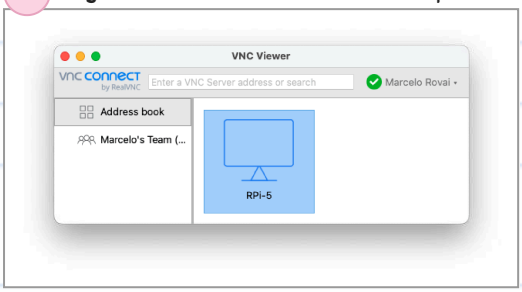
raspi-config: top menu and interface options (example screens)

my notes

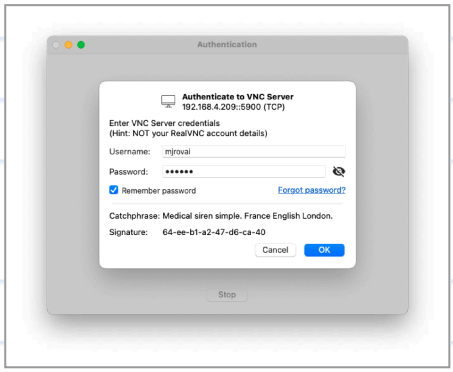
D. CONNECT FROM THE LAPTOP

15. Steps

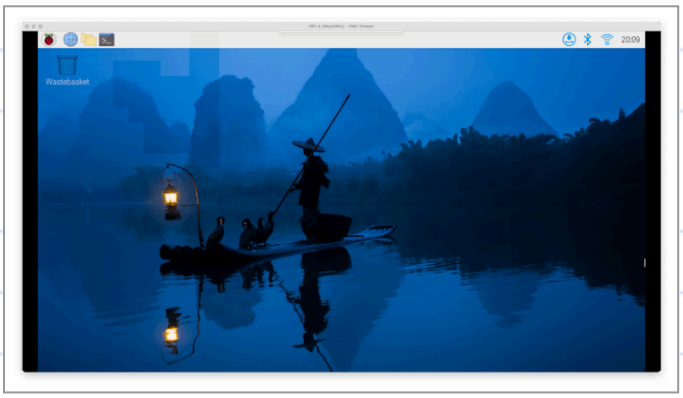
1. Install **RealVNC Viewer** (free) or another VNC viewer.
2. Type the Pi's address in the address bar; accept the identity (fingerprint) message once.
3. Sign in with the **Pi** user name and password (not an account of the VNC company). The desktop appears in a window.



viewer with a saved Pi



sign in with the Pi's account



the Pi desktop in a window (example on a Mac)

my notes

16. A comfortable desktop

- **Resolution:** Preferences -> Screen Configuration (or Display Options in raspi-config).
- **Speed:** on a slow network lower picture quality / colour depth in the viewer; remove the wallpaper.
- **Copy and paste** usually works through the viewer.
- **Close it when finished:** the desktop uses memory that models will need.
- **Never** open VNC's port (5900) to the Internet.

Black or tiny desktop with no monitor? There is no real screen: in `sudo raspi-config` -> Display Options set a resolution (e.g. 1920x1080). The VNC server depends on the OS release (RealVNC Server on older ones; wayvnc possible with the Wayland desktop). If the viewer refuses or shows nothing, raspi-config -> Advanced Options lets me switch between Wayland and X11; X11 with RealVNC is the best-documented pair.

my notes

E. MORE THAN ONE NETWORK

17. A Pi can remember several Wi-Fi networks

- Imager stored one network. Like a phone, a Pi can remember many and join whichever it finds, so the same Pi works at the lab and at home without re-writing the card.
- Current Raspberry Pi OS uses **NetworkManager**, driven by `nmcli`.

```
nmcli connection show   # saved connections
nmcli device status   # devices; wlan0 = Wi-Fi
nmcli device wifi list   # networks in range now
# network in range now:
sudo nmcli device wifi connect "HomeWiFi" password "the-password"
# network NOT in range (prepare a saved profile):
sudo nmcli connection add type wifi con-name home ifname wlan0 ssid "HomeWiFi" \
    wifi-sec.key-mgmt wpa-psk wifi-sec.psk "the-password"
sudo nmcli connection modify home connection.autoconnect yes connection.autoconnect-priority 10
```

`autoconnect-priority 10`	higher number = preferred when several are in range
`sudo nmcli connection delete home`	forget a profile
`sudo nmtui`	menu version: Edit a connection -> Add -> Wi-Fi

Do not cut my own branch: if I am connected over Wi-Fi and switch the Pi to another network, my SSH session ends. Add profiles first, test later, keep a monitor, keyboard or Ethernet cable as a way back.

Older OS (before Bookworm): Wi-Fi lives in `/etc/wpa_supplicant/wpa_supplicant.conf` (one `network={ ssid="..." psk="..." }` block each) or raspi-config's Wireless LAN. Check with `cat /etc/os-release`.

my notes

E. FINDING THE PI; STABLE ADDRESSES

18. When `.local` does not work, find the IP

- **Router page** (often 192.168.1.1 or 192.168.0.1): the list of connected devices shows the host name.
- **From Windows:** `ping raspi-lab.local` shows the address if the name works; `arp -a` lists devices my laptop met recently. Raspberry Pi MAC addresses often start with `b8-27-eb`, `dc-a6-32` or `e4-5f-01` (newer boards use more).
- **With a screen:** plug a monitor in and run `hostname -I`.
- Then put the IP in the `HostName` line of my SSH config.

my notes

19. Keep the address stable

- The router (DHCP) may give the Pi a new address after a reboot.
- Best fix: a **DHCP reservation** (static lease): tell the router to always give this Pi's MAC address the same IP. Prefer this to a fixed address on the Pi: the router stays in charge of the numbers.

my notes

20. Two fallbacks

- **Phone hotspot:** Pi and laptop on the phone's hotspot avoid guest and enterprise restrictions.
- **Ethernet cable straight between laptop and Pi:** after about a minute both sides self-assign `169.254.x.x` addresses and `raspi-lab.local` often works. Useful with no Wi-Fi at all.

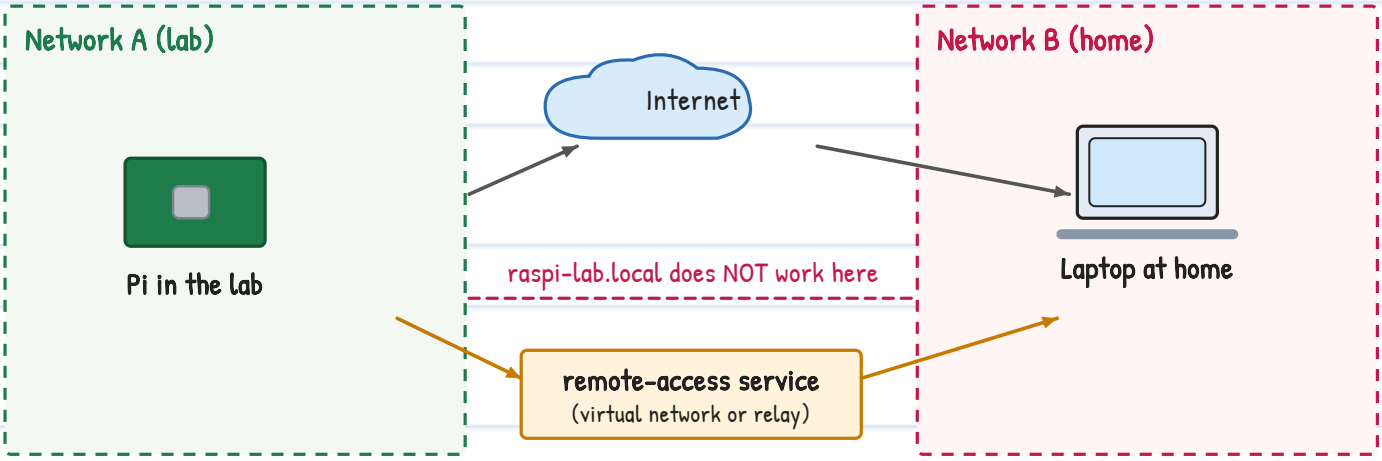
Office and campus networks: many allow only registered devices or block device-to-device traffic. If the Pi will not join, ask the administrator about a guest / IoT network or registering the MAC address. Do not work around the rules.

my notes

F. REACHING A PI IN ANOTHER NETWORK

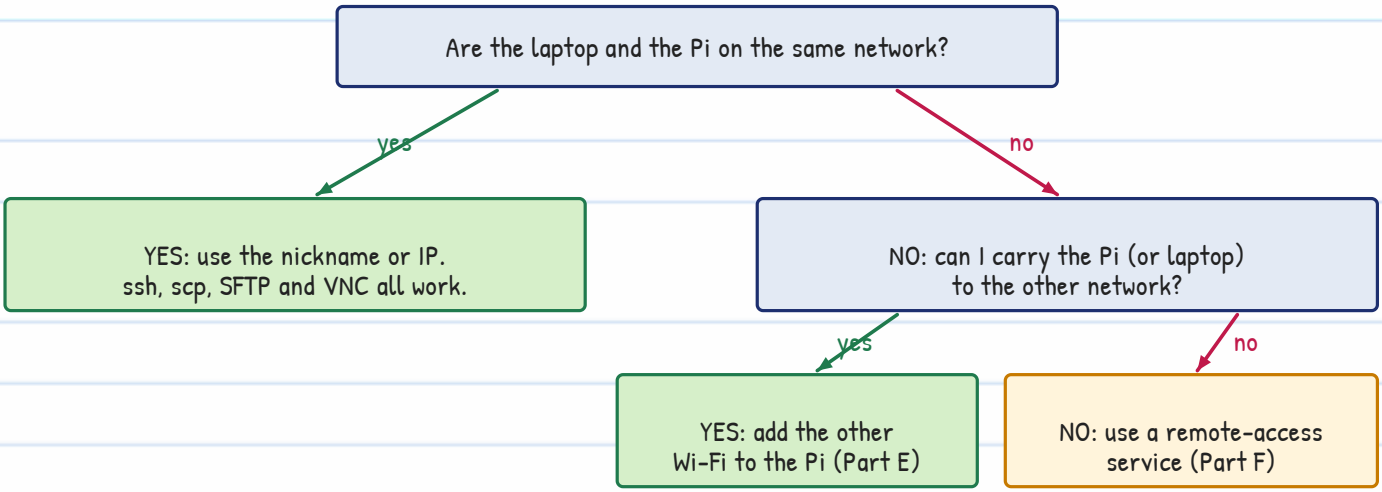
21. Why it is hard

- Laptop at home and Pi in the lab = two private networks. Each router hides its devices behind one public address and refuses unsolicited connections, and `.local` names never leave their network. Something must connect the two sides.



my notes

22. Choosing an approach



my notes

F. THE OPTIONS

23. The options

Option	How it works	Good	Careful
Carry the Pi, add the other Wi-Fi	Pi joins whichever network it finds	simplest	must move the Pi
Private-network service (e.g. Tailscale)	each device installs a small program and signs in with one account; devices get private addresses and names that work from anywhere	works through home and office routers unchanged	software on devices; needs an account; check policy first
Raspberry Pi Connect	the Pi maker's service: remote screen and shell in a browser; needs a recent OS	from the maker; no router changes	follow current official steps; account + Internet
VPN of my organisation	once connected, my laptop acts as if inside that network	no extra Pi software; approved	only if provided
Port forwarding on the router	opens SSH to the whole Internet	-	**avoid**: automated attacks within minutes

my notes

24. Example idea: a private network (Tailscale)

Create a free account; install the program on the laptop and sign in; install it on each Pi and sign in with the same account; then the devices reach each other by the names in the service's device list. Details change: read the vendor's current instructions.

```
# on the Pi (the vendor's installer; read the script first if you can)
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up   # prints a web address; open it and sign in
tailscale ip -4   # the Pi's private address, e.g. 100.x.y.z
```

Piping a script into the shell (`curl … | sh`) runs downloaded code with my privileges: only for sources I trust, and never on a work computer without permission.
Policy first: installing remote-access software on a work laptop, or opening a path into a work network, can break the rules. Ask IT before trying Part F on any work device or network.

my notes

25. A sensible plan for two Pis and two laptops

- Keep each Pi in **one place** and add the other place's Wi-Fi as a second profile, so I can also carry it.
- Nickname every Pi in the SSH config on every laptop; use key login.
- Code and notes in git; big files with scp.
- For access from outside choose **one** approved method; do not combine several.

my notes

G. SECURITY HABITS

26. Habits

Habit	Why and how
Change weak passwords	long, unique password; `passwd`
Keep the system updated	`sudo apt-get update && sudo apt upgrade` regularly
Use keys	a key cannot be guessed over the network; keep the private key private
Never expose SSH or VNC to the Internet	own network or a private connection only
Know who is logged in	`who`, `last`
Keep secrets out of git and screenshots	passwords, Wi-Fi keys, access tokens
Back up my work	git for code and notes; scp for important data; SD cards fail

my notes

27. Optional: turn password login off

Only after key login works from **every** computer I use. A mistake can lock me out of a Pi with no screen.

```
sudo nano /etc/ssh/sshd_config  # set the line: PasswordAuthentication no
sudo systemctl restart ssh
# keep the current SSH window open; open a NEW window and confirm that ssh lab still works
before closing the first
```

my notes

WHEN SOMETHING DOES NOT WORK

28. Troubleshooting

I see	probably	try
`Could not resolve hostname lab`	config file missing / wrong place / has .txt	`dir $env:USERPROFILE\.ssh` ; check the `Host` line
`Bad configuration option`	typo in the config file	indent with spaces; check spelling
`Connection refused`	no SSH server listening, or still booting	wait; enable SSH (raspi-config) or re-write the card with SSH on
`Connection timed out`	different networks, VPN, isolation	same network; VPN off; hotspot; Parts E, F
`Permission denied (publickey)`	key not installed / wrong permissions	repeat B2 step 2; `chmod 700 ~/.ssh`; `chmod 600 ~/.ssh/authorized_keys`
still asks the password	key not appended, other key offered	`ssh -v lab` ; repeat step 2; use `>>` not `>`
`HOST IDENTIFICATION HAS CHANGED`	Pi was re-written	`ssh-keygen -R raspi-lab.local`
`scp: No such file or directory`	wrong path / folder missing	`ssh lab "ls ~"` ; create with `mkdir -p`
scp made a file called `lab`	colon missing	write `lab:~/`
VNC cannot connect	not enabled / wrong address / other network / viewer-server mismatch	enable VNC; `hostname -I` ; Wayland/X11 note
VNC black or tiny	no monitor, no resolution	raspi-config -> Display Options
Pi does not join Wi-Fi 2	typo, country, band, enterprise	`nmcli connection show` , `nmcli device wifi list` ; hotspot to test
`nmcli: command not found`	older OS	see the older-OS note (page 10)

my notes

QUIZ & CHEAT SHEET

29. Check myself (answer here first!)

1. Why is `ssh lab` shorter than the long form? Where does the extra information come from?
2. Private versus public key: which may I share?
3. Two laptops: how do both get key access to one Pi?
4. What does tmux do; which keys detach?
5. In `scp a.txt lab:~/` what is source and destination? How to copy from the Pi to the current folder?
6. Why not put a 500 MB model in git? What instead?
7. When is SSH enough, when VNC?
8. No monitor, VNC shows black or tiny: what to change?
9. How to add a Wi-Fi to a Pi not in range of it?
10. `.local` fails on a guest Wi-Fi: two ways to find the Pi?
11. Two safe ways and one unsafe way to reach a Pi in another network?
12. Three security habits?

my notes

30. Answers

1 the config file maps the nickname to host name and user. 2 private = secret, stays on the laptop; public goes on the Pi and may be shared. 3 a key pair per laptop; append both public keys to the Pi's `authorized_keys`. 4 keeps a terminal session alive on the Pi; Ctrl+b then d. 5 source `a.txt`, destination the Pi's home folder; `scp lab:file .` 6 big files do not suit git: scp / SFTP and `.gitignore`. 7 SSH for commands, installs, scripts; VNC for graphical programs and previews. 8 set a resolution in raspi-config, Display Options. 9 `nmcli connection add type wifi …`. 10 router device list; `arp -a` / `ping`; a monitor with `hostname -I`. 11 safe: private-network service, approved VPN, carrying the Pi; unsafe: port forwarding. 12 strong password, updates, keys, no Internet exposure, backups.

my notes

COMMAND CHEAT SHEET

31. Cheat sheet

Task	Command
connect with a nickname	`ssh lab`
create a key pair	`ssh-keygen -t ed25519`
file to the Pi / from the Pi	`scp file.txt lab:~/` / `scp lab:file.txt .`
a folder	`scp -r folder lab:~/`
one command on the Pi	`ssh lab "hostname -I"`
tmux start / detach / resume	`tmux new -s work` / Ctrl+b d / `tmux attach -t work`
enable VNC	`sudo raspi-config` -> Interface Options -> VNC
Wi-Fi in range / saved	`nmcli device wifi list` / `nmcli connection show`
add a Wi-Fi profile	`sudo nmcli connection add type wifi con-name NAME ifname wlan0 ssid "SSID" wifi-sec.key-mgmt wpa-psk wifi-sec.psk "PASSWORD"`
the Pi's IP	`hostname -I`
update / shut down	`sudo apt-get update && sudo apt upgrade` / `sudo shutdown -h now`
sync code	`git pull` / `git add .` `git commit -m "..."` `git push`

my notes

32. Day 2 is done when...

- [ ] `ssh lab` logs me in with no password
- [ ] a file travelled to the Pi and another came back
- [ ] I saw the Pi desktop in a VNC window
- [ ] the Pi has a second Wi-Fi profile (or I know how to add one)
- [ ] my "Pis and laptops" table is filled in

my notes

GLOSSARY & MY PIS

33. Glossary

Term	Meaning
SSH config file	`~/.ssh/config` : nicknames and options for SSH destinations
key pair	private key (secret) + public key (shareable): identity without a password
`authorized_keys`	file on the Pi listing the public keys allowed to log in
tmux	keeps terminal sessions alive on the Pi
scp / SFTP	copy over SSH on the command line / file transfer with two-pane programs
git, repository	version control; the shared online store of a project
VNC	remote control of a graphical desktop
NetworkManager, nmcli	network service of current Raspberry Pi OS and its command-line tool
DHCP reservation	router setting: same IP for the same device, always
NAT	router technique hiding private addresses behind one public address
link-local address	`169.254.x.x` : self-assigned when no DHCP server answers
VPN / overlay network	private network over the Internet; distant devices behave as if together
port forwarding	router rule exposing a port to the Internet (not for SSH)

my notes

34. My Pis and laptops

	Pi 1	Pi 2	Laptop 1	Laptop 2
board / RAM				
host name				
SSH nickname				
where it lives				
IP (can change)				
Wi-Fi saved				
key added? VNC on?				

my notes

INSTRUCTOR NOTES

35. A 60-minute lab

10 min the rough edges; demonstrate `ssh lab` versus the long form. **15 min** students create the config file and a key. **15 min** scp both ways and an SFTP program. **15 min** VNC. **5 min** where networks break.

- **Typical mistakes:** config saved as `config.txt`; colon missing in scp; the private key copied instead of the public key; `>` instead of `>>` when adding a key (overwrites other keys); VNC with no resolution set.
- **Mini-assessment:** `ssh lab` works without a password, and a screenshot of `ls -l` and the scp lines for a file copied both ways.
- **Policy:** warn students not to install remote-access software on school or work computers without permission.

my notes

36. My own notes, ideas, questions

my notes

Tomorrow, Day 3: the Linux command line.

ACKNOWLEDGMENTS & REFERENCES

This course is a compilation: it combines material from the sources below with new explanations, examples, diagrams, exercises and comments written for this course. The original authors are gratefully acknowledged.

[1] Rovai, M. (2025). Edge AI Engineering: Hands-on with the Raspberry Pi. Open textbook for the course IESTI05, Federal University of Itajubá (UNIFEI), Brazil. Version of 23 October 2025. Online edition: https://mjrovai.github.io/EdgeML_Made_Ease_ebook/

[2] Rovai, M. (2026). IESTI05 - Edge AI: Edge Machine Learning Systems Engineering. Course slides, notebooks and video lectures (second semester 2026), UNIFEI, Brazil. Course videos listed by the author: Class 1 (course introduction) https://youtu.be/QAb3MFHwe78 ; Class 2 (Introduction to Linux) https://youtu.be/QkIHILtl0h4 ; Class 3 (Raspberry Pi setup walkthrough) https://youtu.be/YtjsM36LzVE (lectures in Portuguese).

[3] Rovai, M. (2026). IESTI01 - TinyML: Machine Learning for Embedding Devices. Course materials (first semester 2026), UNIFEI, Brazil. https://github.com/Mjrovai/UNIFEI-IESTI01-TinyML

[4] Rovai, M. TinyML4D: lectures, books, papers and tutorials for embedded machine-learning education. https://github.com/Mjrovai/TinyML4D

[5] Raspberry Pi Ltd. Raspberry Pi documentation (operating system, remote access, Raspberry Pi Connect, hardware). https://www.raspberrypi.com/documentation/

[6] OpenSSH project. OpenSSH: the SSH client and server used by Windows, macOS, Linux and Raspberry Pi OS (manual pages ssh, ssh_config, scp, ssh-keygen). https://www.openssh.com/

[7] FileZilla Project. FileZilla Client, a free SFTP program. https://filezilla-project.org/

[8] RealVNC. VNC Viewer and VNC Server. https://www.realvnc.com/

[9] Tailscale Inc. Tailscale documentation (private network between your own devices). https://tailscale.com/

The screenshots of the scp transfer, FileZilla, raspi-config and the VNC viewer are adapted from reference [1]. All other figures and the animation were created for this course.

my notes